ANMOL JANAMATTI

2JI23CS022

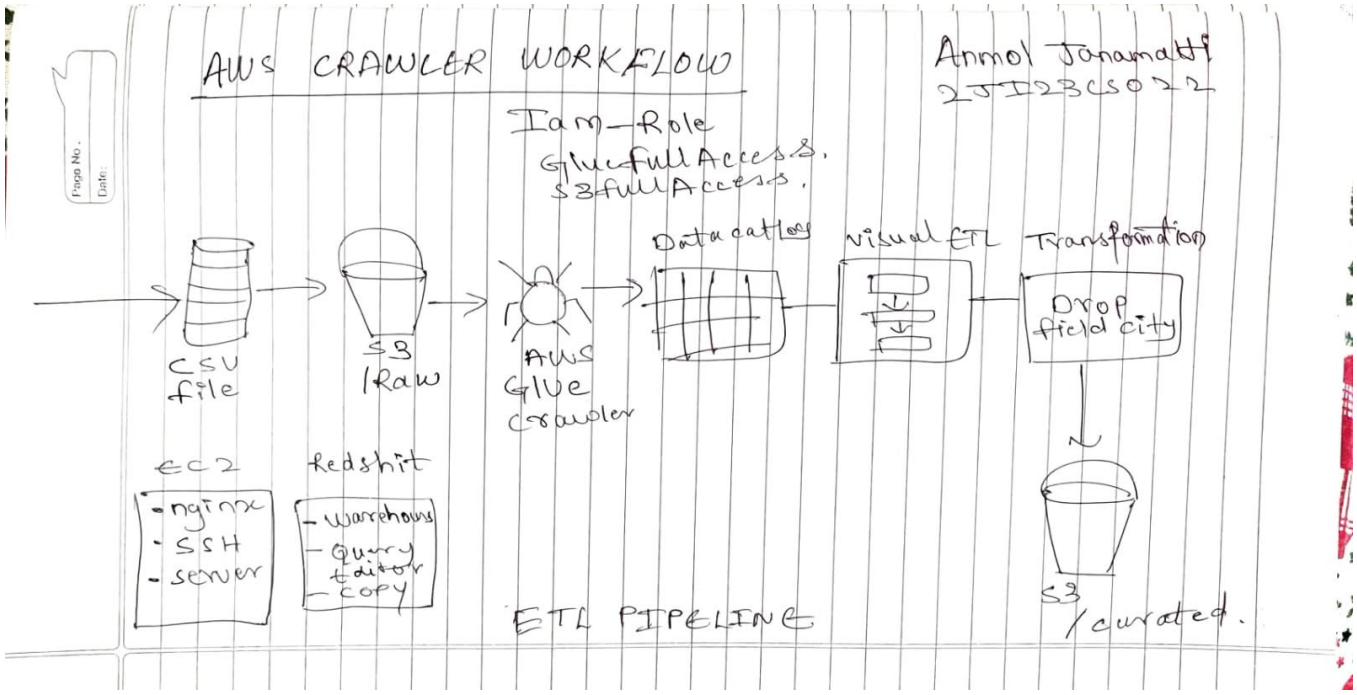
Jain College of Engineering

2027

Date : 11/10/2026

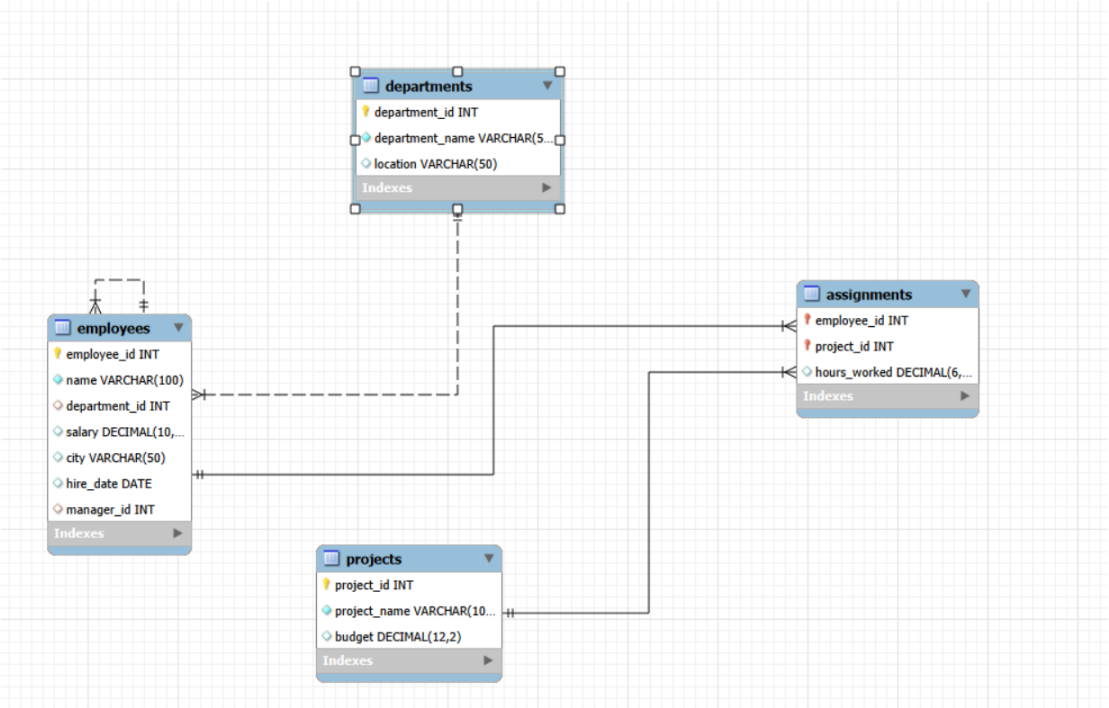
TITLE : AWS Cloud & Data Engineering

My ETL Pipeline

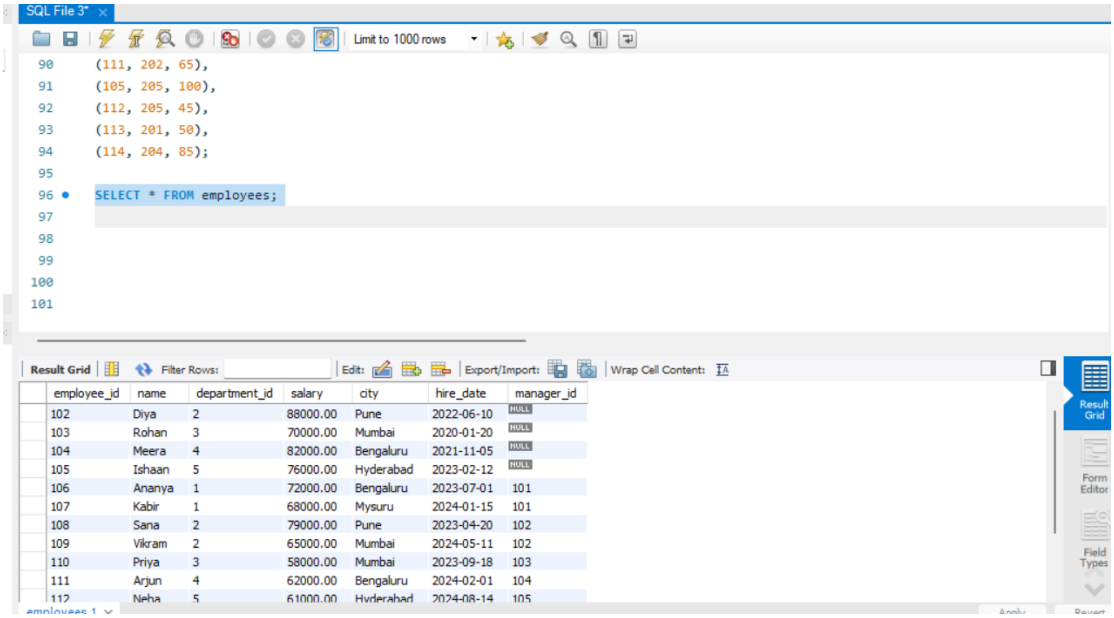


SQL Practice:

ER diagram of the database (employees, departments, projects and assignments tables)



Query 1: Display all the records of the employees table.



Query 2: Display the employees earning more than 70000, highest salary first.

```
97
98 •  SELECT *
99    FROM employees
100   WHERE salary > 70000
101   ORDER BY salary DESC;
102
103
104
105
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_id	name	department_id	salary	city	hire_date	manager_id
101	Aarav	1	95000.00	Bengaluru	2021-03-15	NULL
102	Diya	2	88000.00	Pune	2022-06-10	NULL
114	Tara	2	88000.00	Pune	2022-10-08	102
104	Meera	4	82000.00	Bengaluru	2021-11-05	NULL
108	Sana	2	79000.00	Pune	2023-04-20	102
105	Ishaan	5	76000.00	Hyderabad	2023-02-12	NULL
106	Ananya	1	72000.00	Bengaluru	2023-07-01	101
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query 3: Find the average salary of each department.

```
103 •  SELECT department_id, AVG(salary) AS avg_salary
104    FROM employees
105    GROUP BY department_id;
106
107
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department_id	avg_salary
NULL	50000.000000
1	75750.000000
2	80000.000000
3	64000.000000
4	72000.000000
5	68500.000000

Query 4: Find the departments whose average salary is above 75000.

```
106
107 •  SELECT department_id, AVG(salary) AS avg_salary
108    FROM employees
109    GROUP BY department_id
110    HAVING AVG(salary) > 75000;
111
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department_id	avg_salary
1	75750.000000
2	80000.000000

Query 5: List each employee with the name of their department (INNER JOIN).

```
111
112 •  SELECT e.name, d.department_name
113    FROM employees e
114    INNER JOIN departments d
115        ON e.department_id = d.department_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	department_name
Aarav	Engineering
Ananya	Engineering
Kabir	Engineering
Kiran	Engineering
Diya	Data Analytics
Sana	Data Analytics
Vikram	Data Analytics
Tara	Data Analytics
Rohan	Human Resources
Priya	Human Resources
Meera	Finance

Result 5 | Read Only

Result Grid | Form Editor | Field Types

Query 6: List all employees with their department, including those who have no department (LEFT JOIN).

```
116
117 •   SELECT e.name, d.department_name
118     FROM employees e
119     LEFT JOIN departments d
120         ON e.department_id = d.department_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	department_name
Ananya	Engineering
Kabir	Engineering
Sana	Data Analytics
Vikram	Data Analytics
Priya	Human Resources
Arjun	Finance
Neha	Marketing
Kiran	Engineering
Tara	Data Analytics
Dev	NULL

Query 7: Show each employee with the name of their manager (self join).

```
121
122 •   SELECT
123         e.name AS employee_name,
124         m.name AS manager_name
125     FROM employees e
126     LEFT JOIN employees m
127         ON e.manager_id = m.employee_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_name	manager_name
Ananya	Aarav
Kabir	Aarav
Sana	Diya
Vikram	Diya
Priya	Rohan
Arjun	Meera
Neha	Ishaan
Kiran	Aarav
Tara	Diya
Dev	NULL

Query 8: Find the employees earning more than the company average (subquery).

```
129 •       SELECT name, salary
130     FROM employees
131     WHERE salary > (
132         SELECT AVG(salary)
133         FROM employees
134     );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary
Aarav	95000.00
Diya	88000.00
Meera	82000.00
Ishaan	76000.00
Sana	79000.00
Tara	88000.00

Query 9: Find the employees earning more than the company average using a CTE.

```
135
136 • WITH company_avg AS (
137         SELECT AVG(salary) AS avg_salary
138         FROM employees
139     )
140     SELECT name, salary
141     FROM employees
142     WHERE salary > (
143         SELECT avg_salary
144         FROM company_avg
145     );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary
Aarav	95000.00
Diya	88000.00
Meera	82000.00
Ishaan	76000.00
Sana	79000.00
Tara	88000.00

Query 10: Find the departments with an average salary above 70000 using two CTEs.

```
146
147  WITH dept_stats AS (
148      SELECT department_id,
149             COUNT(*) AS total_employees,
150             AVG(salary) AS avg_salary
151      FROM employees
152      WHERE department_id IS NOT NULL
153      GROUP BY department_id
154  ),
155  high_salary_depts AS (
156      SELECT *
157      FROM dept_stats
158      WHERE avg_salary > 70000
159  )
160  SELECT *
161  FROM high_salary_depts;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department_id	total_employees	avg_salary
1	4	75750.000000
2	4	80000.000000
4	2	72000.000000

Query 11: Number the employees of each department by salary using ROW_NUMBER().

```
SELECT
    name,
    department_id,
    salary,
    ROW_NUMBER() OVER (
        PARTITION BY department_id
        ORDER BY salary DESC
    ) AS row_num
FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	department_id	salary	row_num
Dev	NULL	50000.00	1
Aarav	1	95000.00	1
Ananya	1	72000.00	2
Kabir	1	68000.00	3
Kiran	1	68000.00	4
Diya	2	88000.00	1
Tara	2	88000.00	2
Sana	2	79000.00	3
Vikram	2	65000.00	4
Rohan	3	70000.00	1
Priya	3	58000.00	2

Result 12

Query 12: Find the top two salary ranks in each department using DENSE_RANK().

```
174  WITH ranked_employees AS (
175      SELECT
176          name,
177          department_id,
178          salary,
179          DENSE_RANK() OVER (
180              PARTITION BY department_id
181              ORDER BY salary DESC
182          ) AS salary_rank
183      FROM employees
184      WHERE department_id IS NOT NULL
185  )
186  SELECT name, department_id, salary, salary_rank
187  FROM ranked_employees
188  WHERE salary_rank <= 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

name	department_id	salary	salary_rank
Aarav	1	95000.00	1
Ananya	1	72000.00	2
Diya	2	88000.00	1
Tara	2	88000.00	1
Sana	2	79000.00	2
Rohan	3	70000.00	1

Query 13: Compare each salary with the previous higher salary in the department using LAG().

```
191    SELECT
192        name,
193        department_id,
194        salary,
195        LAG(salary) OVER (
196            PARTITION BY department_id
197            ORDER BY salary DESC, employee_id
198        ) AS previous_salary,
199        LAG(salary) OVER (
200            PARTITION BY department_id
201            ORDER BY salary DESC, employee_id
202        ) - salary AS salary_difference
203    FROM employees
204    WHERE department_id IS NOT NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	department_id	salary	previous_salary	salary_difference
Aarav	1	95000.00	NULL	NULL
Ananya	1	72000.00	95000.00	23000.00
Kabir	1	68000.00	72000.00	4000.00
Kiran	1	68000.00	68000.00	0.00
Diya	2	88000.00	NULL	NULL
Tara	2	88000.00	88000.00	0.00
Sana	2	79000.00	88000.00	9000.00
Vikram	2	65000.00	79000.00	14000.00

Query 14: Group the employees into High, Medium and Low salary bands using CASE.

```
SELECT
    CASE
        WHEN salary >= 80000 THEN 'High'
        WHEN salary >= 65000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_band,
    COUNT(*) AS total_employees
FROM employees
GROUP BY salary_band
ORDER BY total_employees DESC;
```

Result Grid | Filter Rows: | Export:

salary_band	total_employees
Medium	7
High	4
Low	4

Query 15: Find the number of employees hired and the average salary for each year.

```
218    SELECT
219        YEAR(hire_date) AS hiring_year,
220        COUNT(*) AS employees_hired,
221        ROUND(AVG(salary), 2) AS average_salary
222    FROM employees
223    WHERE hire_date IS NOT NULL
224    GROUP BY YEAR(hire_date)
225    ORDER BY hiring_year;
```

Result Grid | Filter Rows: | Export: | Wrap

hiring_year	employees_hired	average_salary
2020	1	70000.00
2021	2	88500.00
2022	2	88000.00
2023	5	70600.00
2024	4	64000.00
2025	1	50000.00

Query 16: Find the number of employees and the total hours worked for each project.

```
226
227     SELECT
228         p.project_name,
229         COUNT(a.employee_id) AS employees_assigned,
230         SUM(a.hours_worked) AS total_hours
231     FROM projects p
232     JOIN assignments a
233         ON p.project_id = a.project_id
234     GROUP BY p.project_id, p.project_name
235     ORDER BY total_hours DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

project_name	employees_assigned	total_hours
Data Warehouse	5	380.00
Cloud Migration	4	330.00
HR Automation	2	185.00
Sales Dashboard	2	155.00
Website Redesign	2	145.00

Linux Practice:

ps : Display the running processes of the current terminal

```
anmol@Victus:~$ ps
    PID TTY          TIME CMD
    414 pts/0    00:00:00 bash
   1024 pts/0    00:00:00 ps
anmol@Victus:~$
```

top : Display live process and memory usage

```
anmol@Victus:~$ top
top - 05:46:13 up 0 min,  1 user,  load average: 0.12, 0.04, 0.01
Tasks:  33 total,   1 running,  32 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7757.2 total,   6797.9 free,    533.0 used,    577.5 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7224.2 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   21760  12244   9300 S   0.0   0.2   0:00.43 systemd
      2 root      20   0    3060   1664   1664 S   0.0   0.0   0:00.00 init-systemd(Ub
      6 root      20   0    3060   1792   1792 S   0.0   0.0   0:00.00 init
     42 root      19  -1   42164  15616  14592 S   0.0   0.2   0:00.10 systemd-journal
     89 root      20   0   25544   6528   4864 S   0.0   0.1   0:00.11 systemd-udevd
     98 root      20   0  152936   1540   1408 S   0.0   0.0   0:00.00 snapfuse
```

mkdir and ls : Create a directory and list it

```
anmol@Victus:~$ mkdir test_folder
anmol@Victus:~$ ls
hcltest  ok.js  snap  test  test_folder
anmol@Victus:~$
```

ls -a : List all files including hidden files

```
anmol@Victus:~$ ls -a
.   .aws    .bash_history  .bashrc  .config  .gitconfig  .landscape  .local       .npm  .profile                 .zshrc   ok.js  test
..  .azure  .bash_logout   .cache   .docker  .hermes     .lesshst    .motd_shown  .nvm  .sudo_as_admin_successful  hcltest  snap   test_folder
anmol@Victus:~$
```

ls -hla : Long listing with permissions and human-readable sizes

```
anmol@Victus:~$ ls -hla
total 88K
drwxr-x--- 14 anmol anmol 4.0K Oct 11 05:46 .
drwxr-xr-x  3 root  root  4.0K Sep  9  2025 ..
lrwxrwxrwx  1 anmol anmol   23 May 28 16:21 .aws -> /mnt/c/Users/Anmol/.aws
lrwxrwxrwx  1 anmol anmol   25 May 28 16:21 .azure -> /mnt/c/Users/Anmol/.azure
-rw-------  1 anmol anmol 3.1K Oct  7 11:49 .bash_history
-rw-r--r--  1 anmol anmol  220 Sep  9  2025 .bash_logout
-rw-r--r--  1 anmol anmol 4.0K Oct  7 11:43 .bashrc
```

touch and ls : Create an empty file and list it

```
anmol@Victus:~$ touch test.py
anmol@Victus:~$ ls
hcltest  ok.js  snap  test  test.py  test_folder
anmol@Victus:~$
```

man chmod : Manual page of the chmod command

```
CHMOD(1)                                  User Commands                                  CHMOD(1)

NAME
       chmod - change file mode bits

SYNOPSIS
       chmod [OPTION]... MODE[,MODE]... FILE...
       chmod [OPTION]... OCTAL-MODE FILE...
       chmod [OPTION]... --reference=RFILE FILE...

DESCRIPTION
       This  manual page documents the GNU version of chmod.  chmod changes the file mode bits of each given file according to mode, which can be either
       a symbolic representation of changes to make, or an octal number representing the bit pattern for the new mode bits.

       The format of a symbolic mode is [ugoa...][[-+=][perms...]...], where perms is either zero or more letters from the set rwxXst, or a single  let-
       ter from the set ugo.  Multiple symbolic modes can be given, separated by commas.

       A  combination  of the letters ugoa controls which users' access to the file will be changed: the user who owns it (u), other users in the file's
       group (g), other users not in the file's group (o), or all users (a).  If none of these are given, the effect is as if (a) were given,  but  bits
       that are set in the umask are not affected.
```

cd : Change the current directory

```
anmol@Victus:~$ ls
hcltest  ok.js  snap  test  test.py  test_folder
anmol@Victus:~$ cd hcltest/
anmol@Victus:~/hcltest$
```

cat : Display the contents of a file

```
node_modules  package-lock.json  package.json  server.js
anmol@Victus:~/test$ cat server.js
let express = require('express')
let app = express();

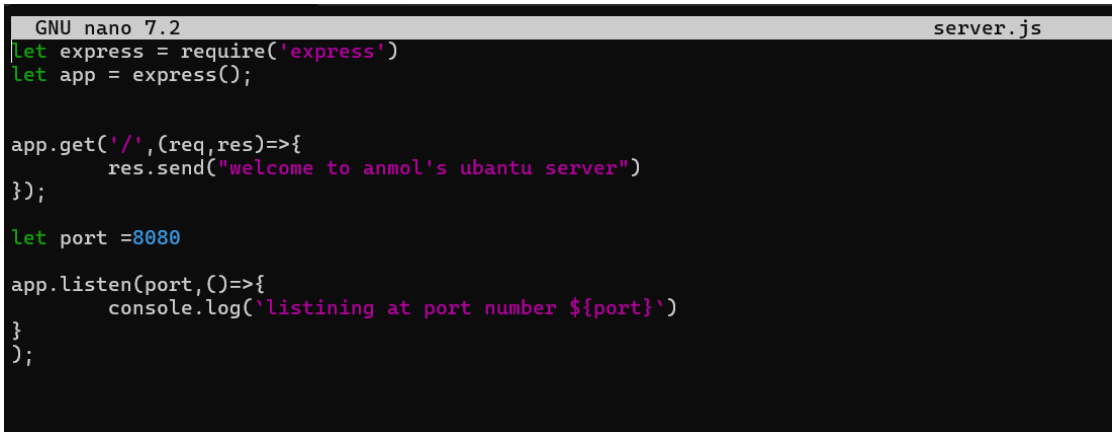

app.get('/',(req,res)=>{
        res.send("welcome to anmol's ubantu server")
});

let port =8080

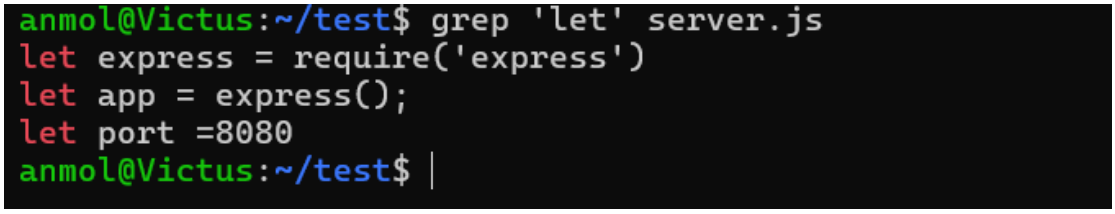
app.listen(port,()=>{
        console.log(`listining at port number ${port}`)
}
);

anmol@Victus:~/test$
```

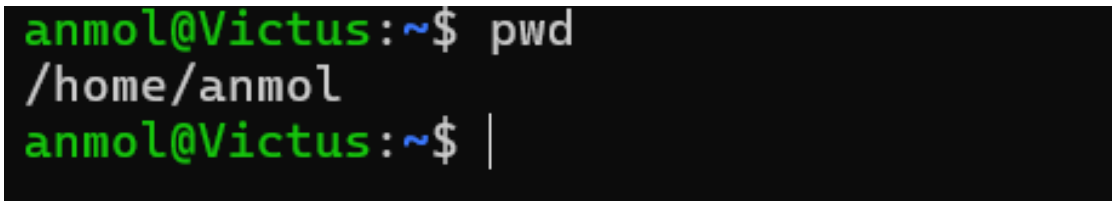
nano : Open the file in the nano editor



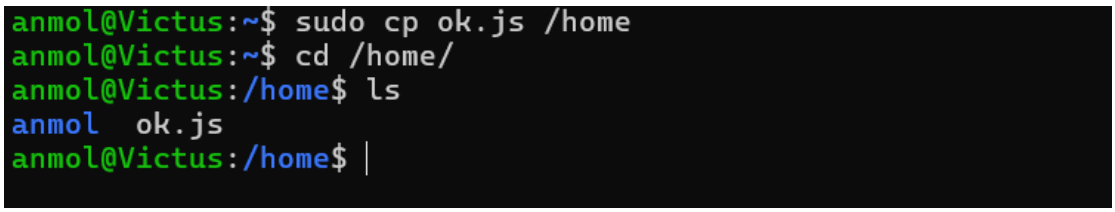
grep : Search for a word in a file



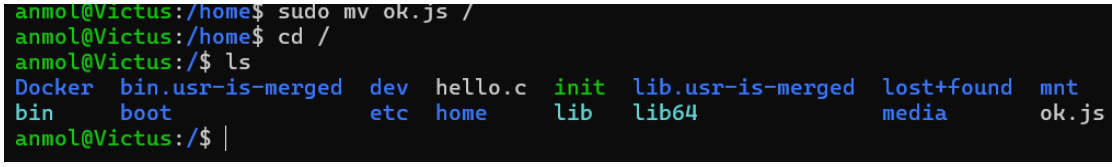
pwd : Show the present working directory



sudo cp : Copy a file to /home

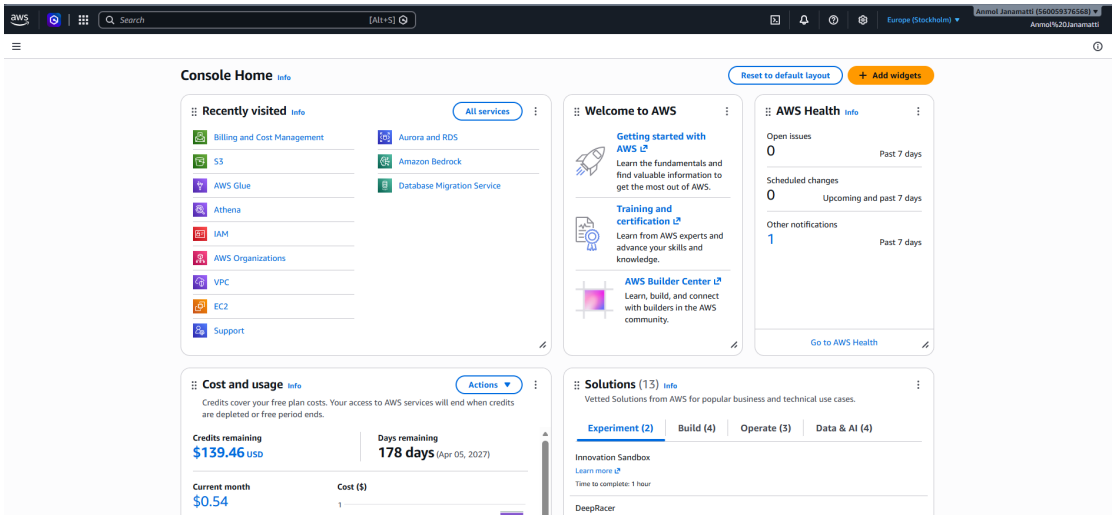


sudo mv : Move a file to the root directory



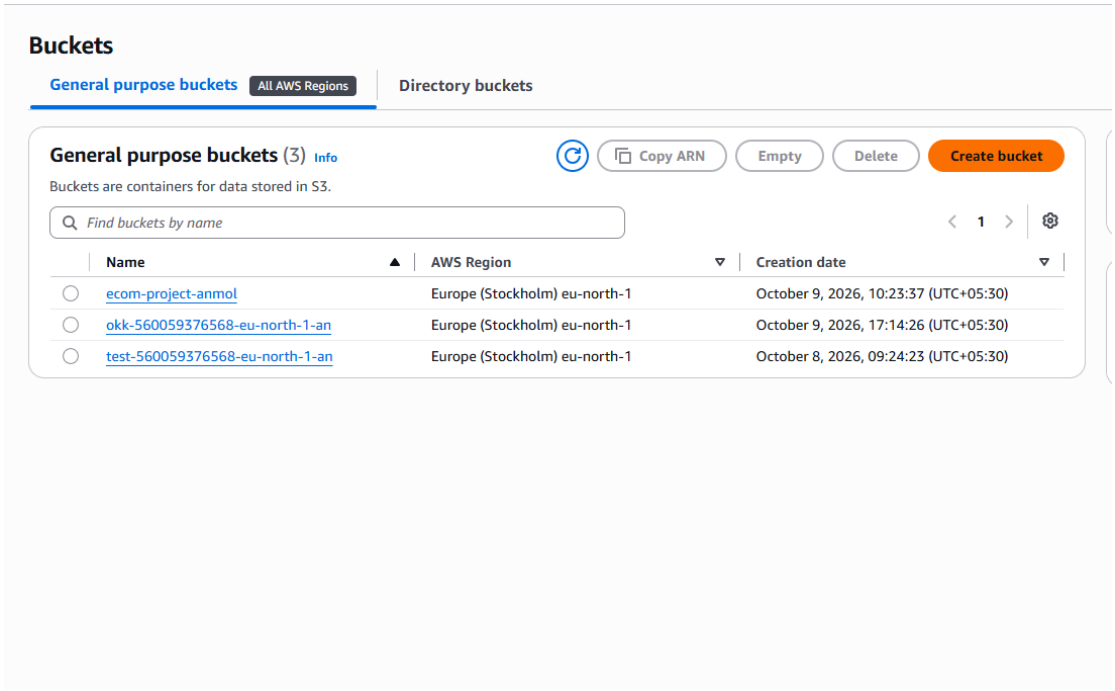
AWS Account / Region:

AWS Console Home with the selected region Europe (Stockholm)

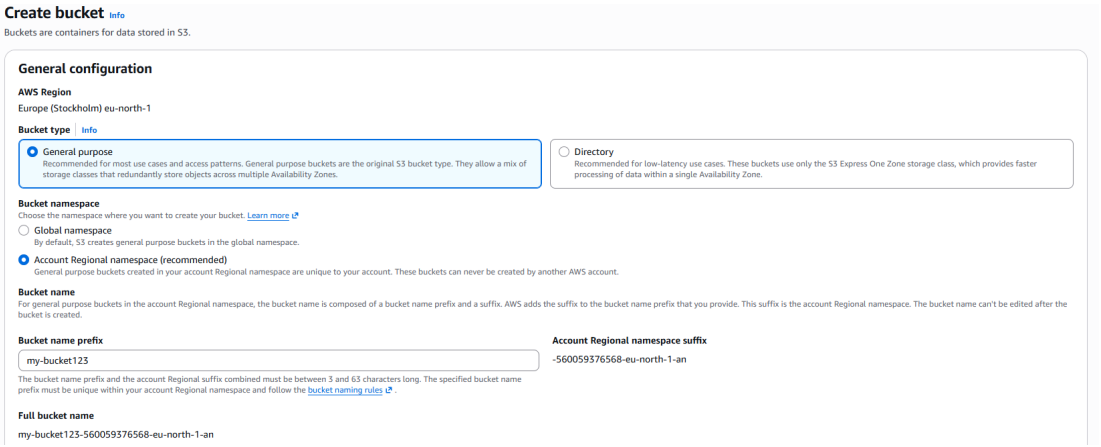


Amazon S3:

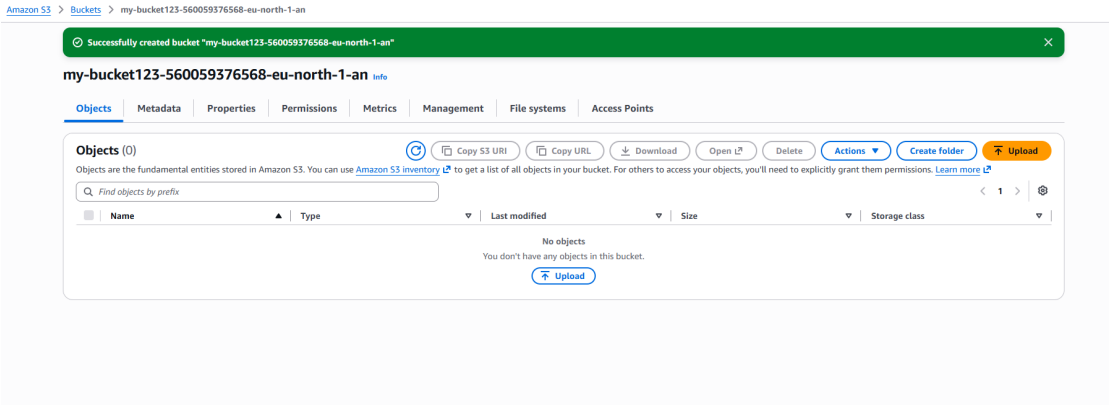
Buckets page of Amazon S3 (Europe (Stockholm) eu-north-1)



Create bucket: general purpose bucket in eu-north-1 with the bucket name my-bucket123-560059376568-eu-north-1-an

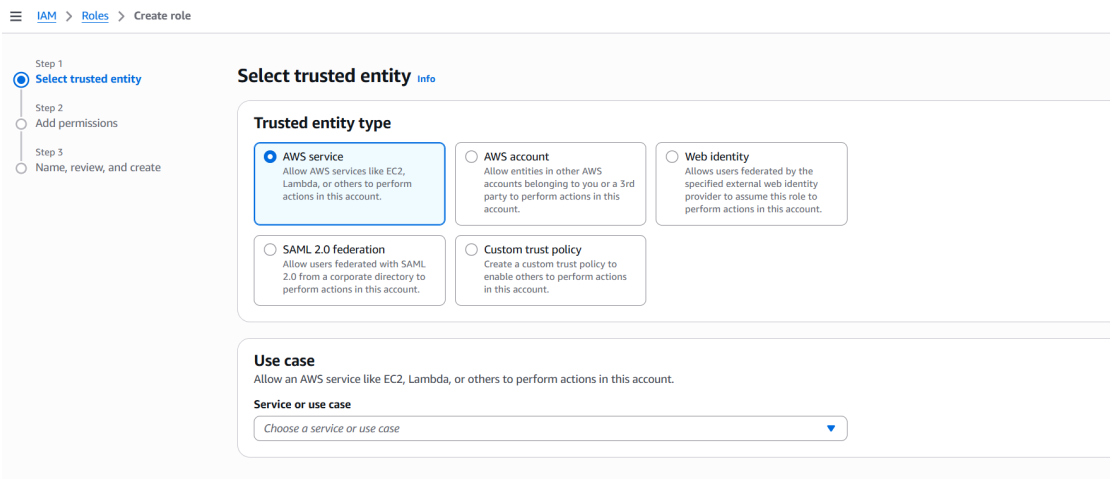


Bucket is created successfully and is empty with 0 objects

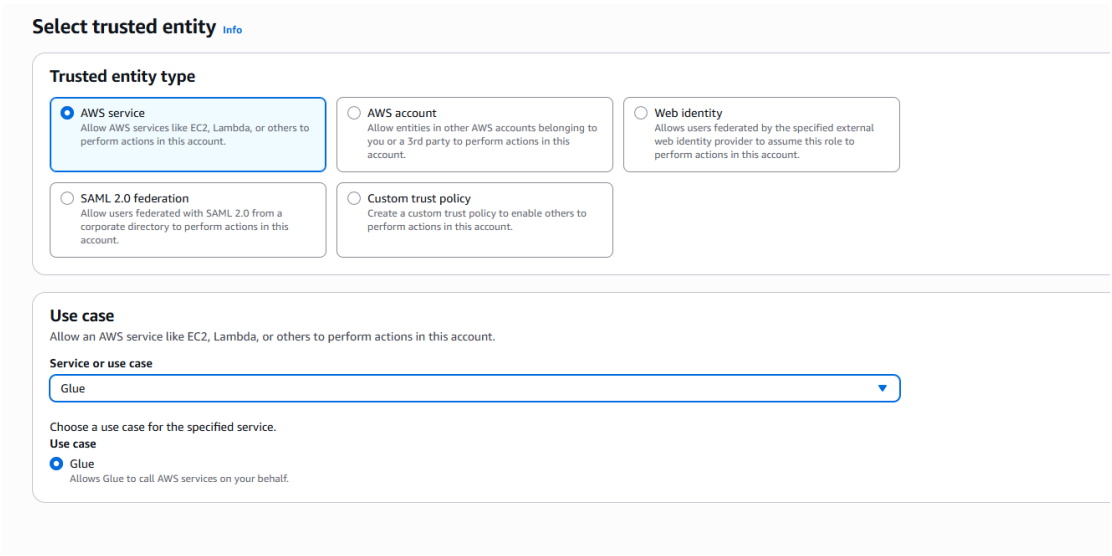


IAM:

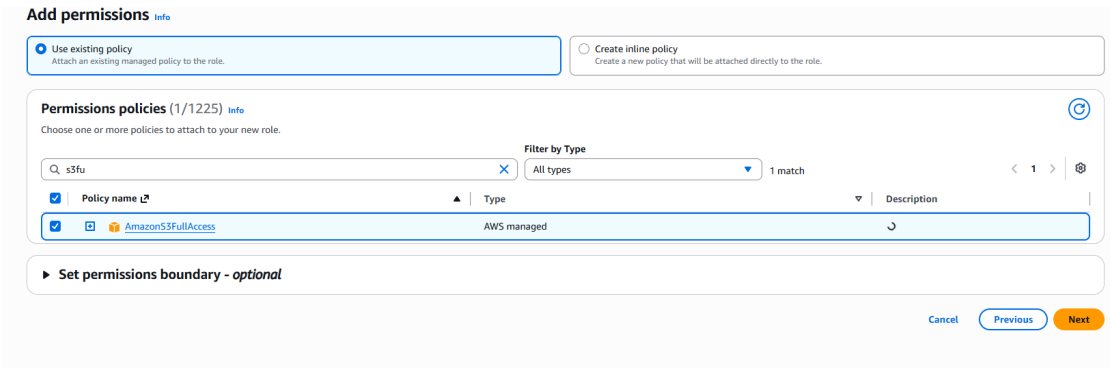
Select trusted entity as AWS service



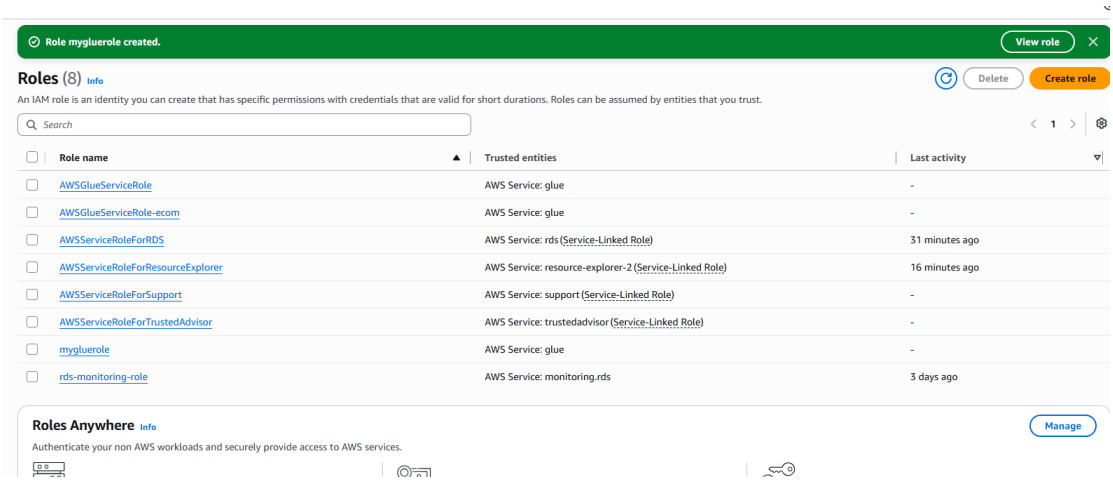
Use case selected as Glue



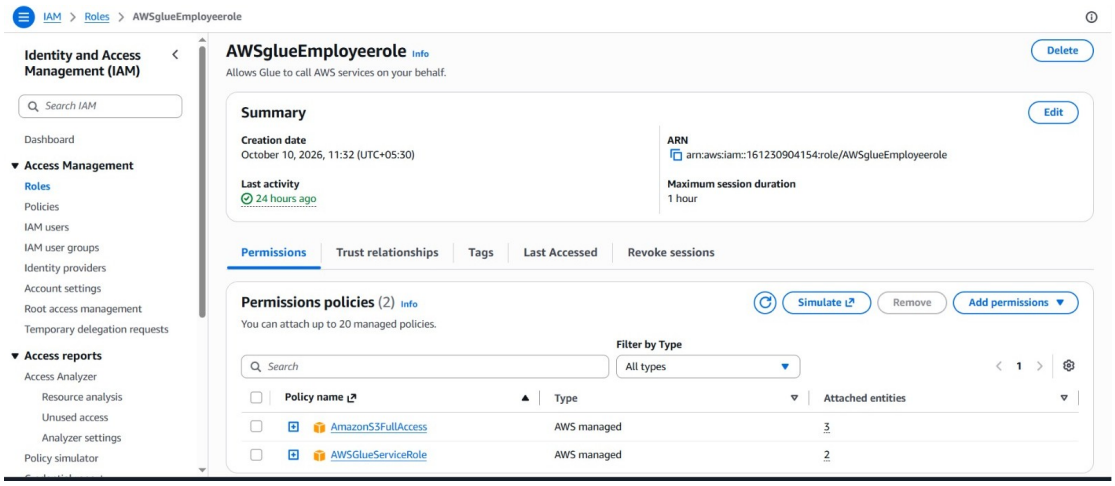
Add permissions: AmazonS3FullAccess policy selected



Role mygluerole created

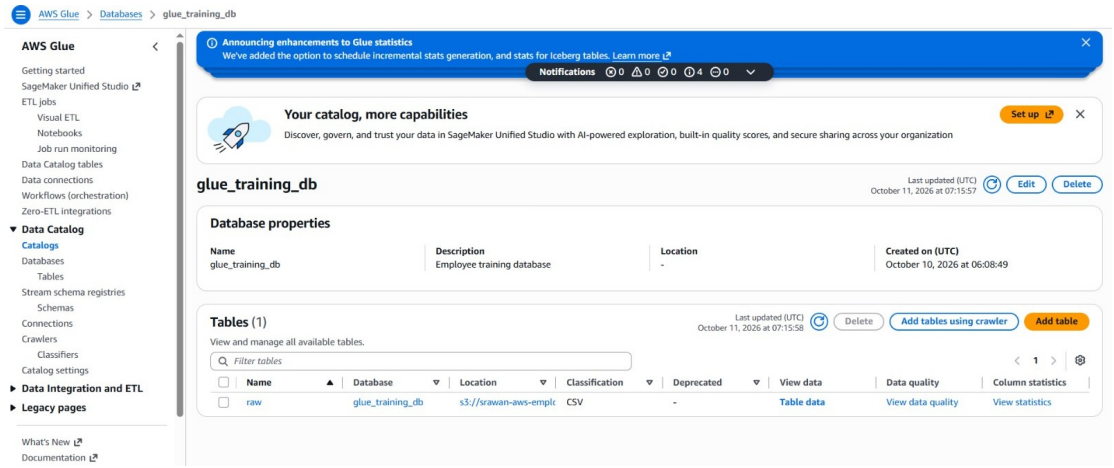


IAM role used for AWS Glue: AWSglueEmployeerole with the AmazonS3FullAccess and AWSGlueServiceRole policies



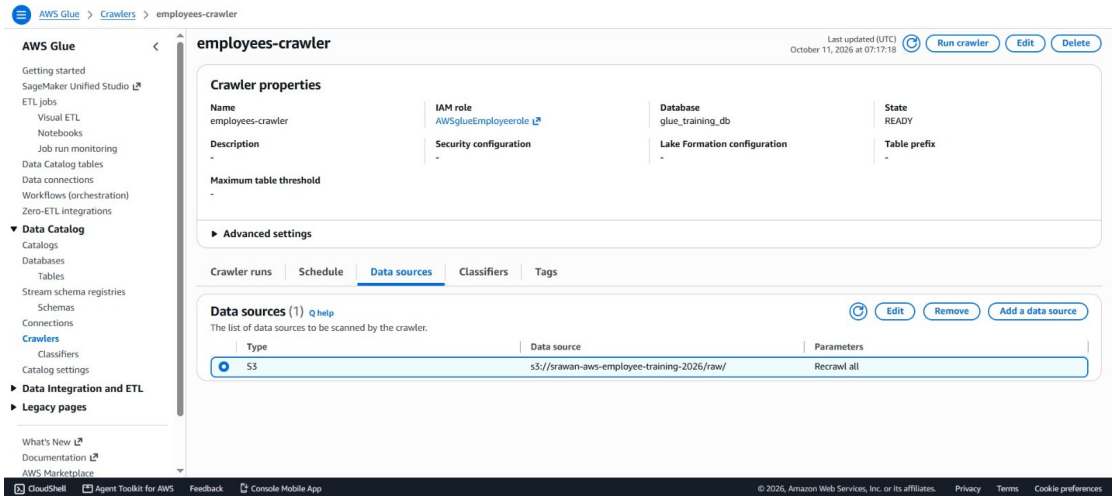
AWS Glue Database:

AWS Glue Data Catalog: database glue_training_db with its properties and tables

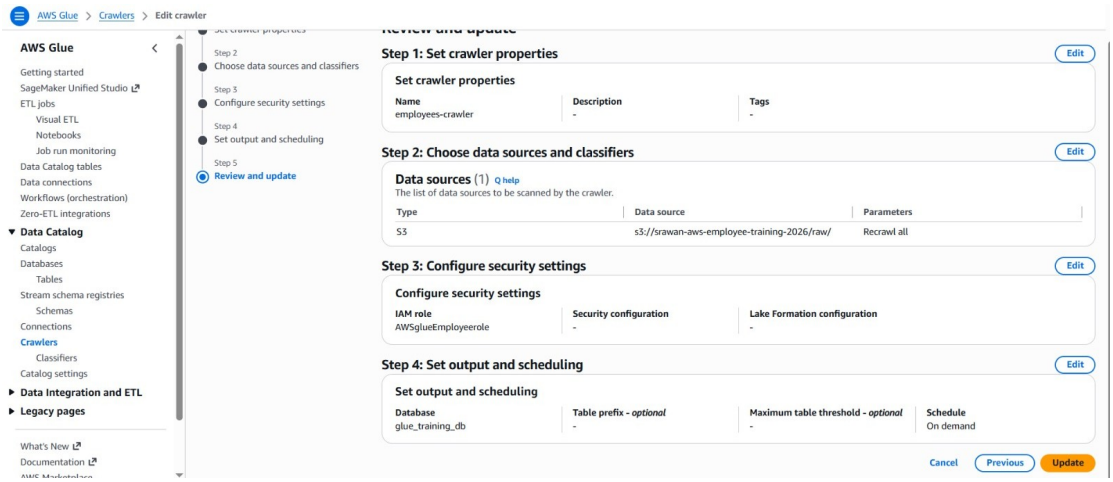


Glue Crawler:

Crawler employees-crawler: S3 data source (raw/ folder), IAM role AWSglueEmployeerole and target database glue_training_db

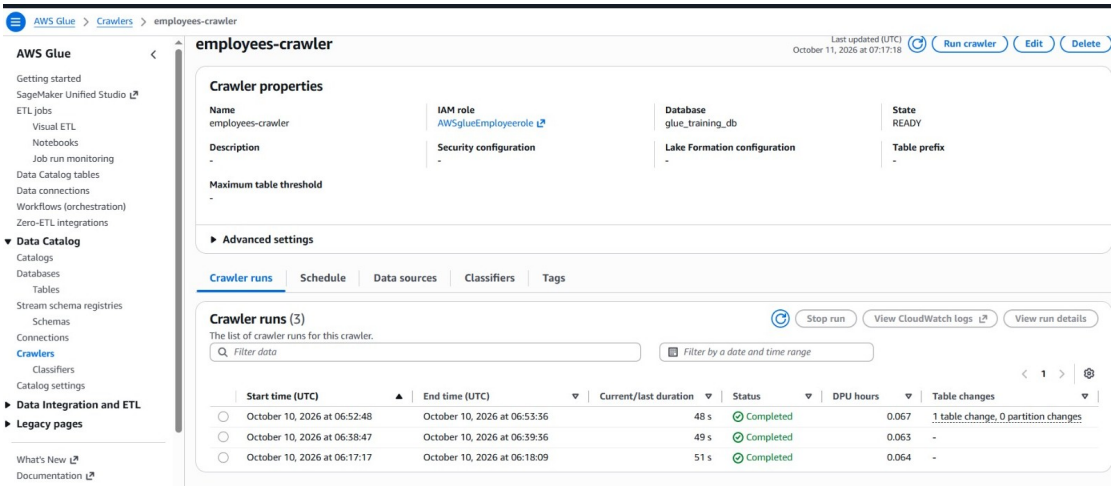


Crawler configuration: review of the crawler properties, data source, security settings and output (schedule: on demand)



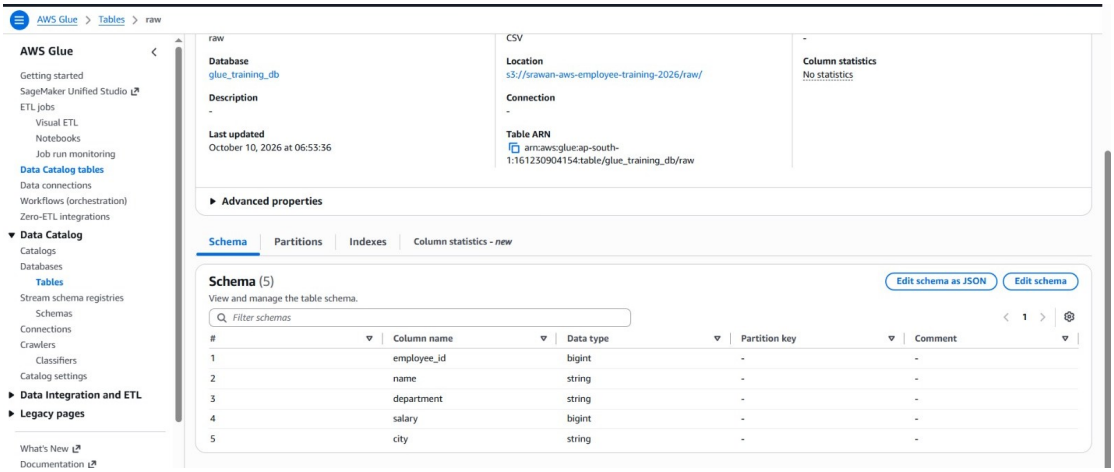
Crawler Execution:

Crawler execution: the crawler runs show the status Completed and the crawler state is READY



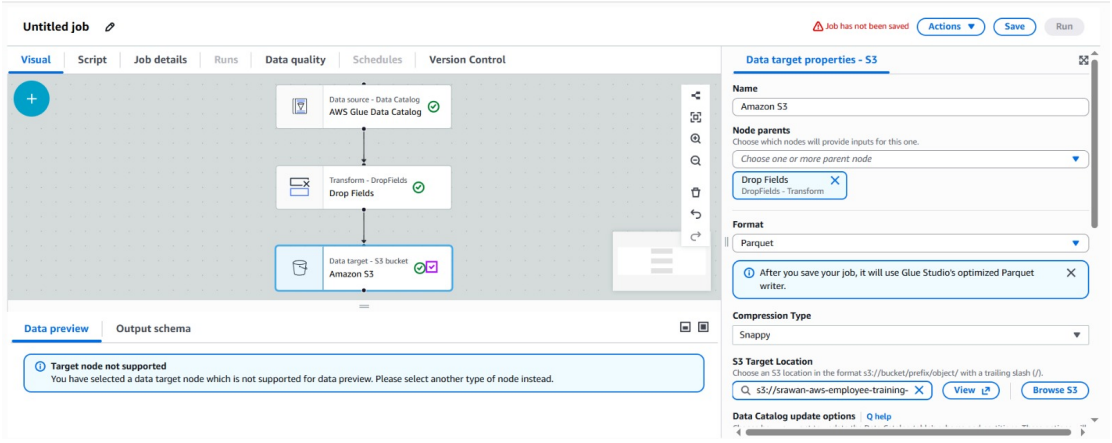
Glue Data Catalog:

Table raw created by the crawler in glue_training_db, with its schema and data types



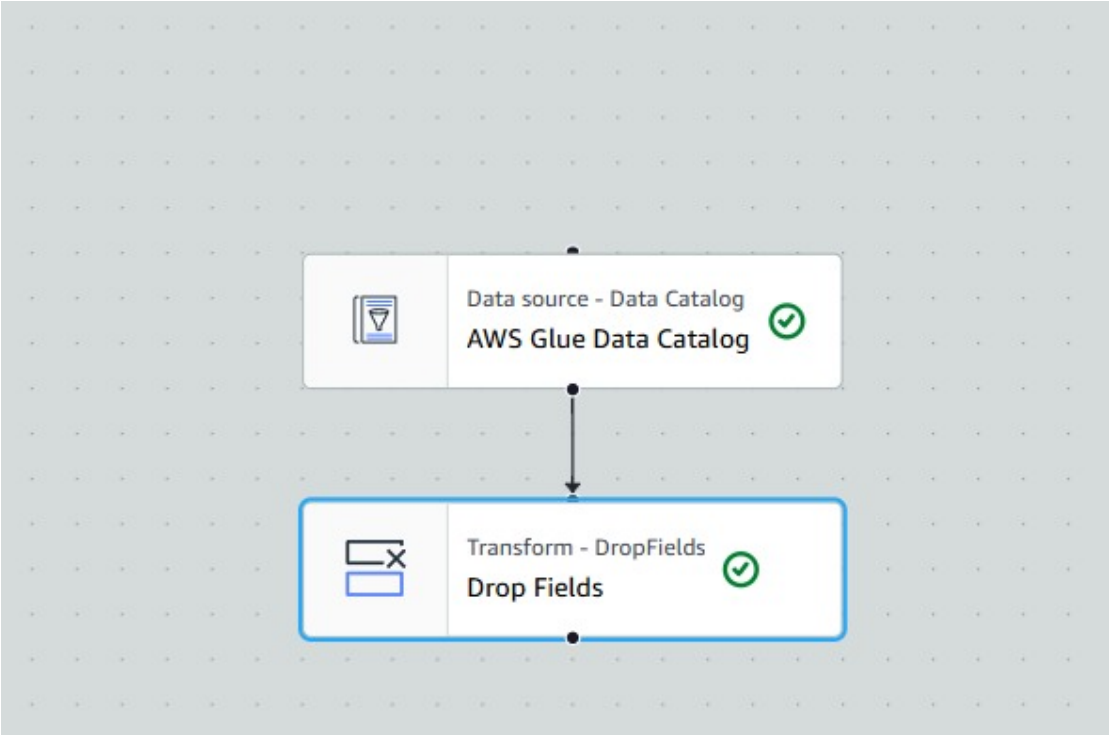
Glue Visual ETL:

Visual ETL job: Data Catalog source, Drop Fields transform and Amazon S3 target



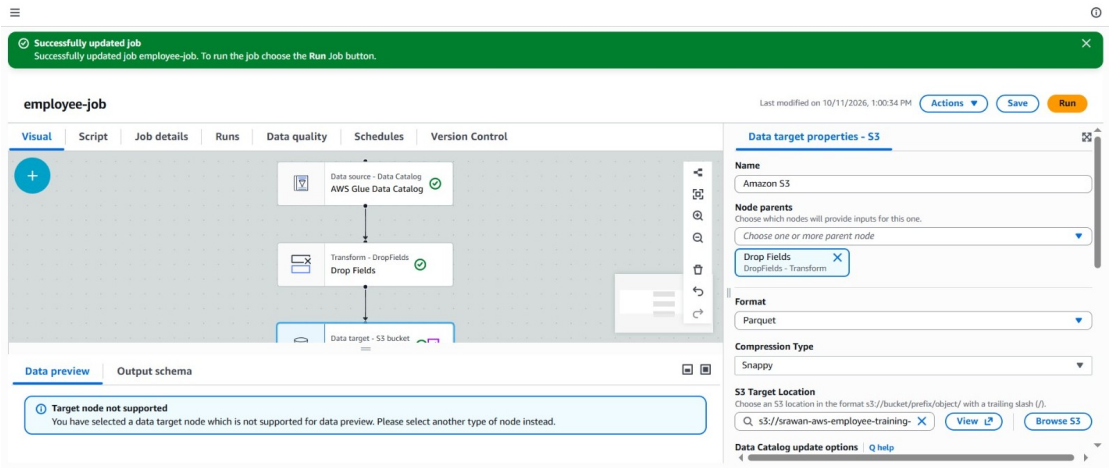
Glue Transformation:

Transformation: Drop Fields node connected to the AWS Glue Data Catalog source



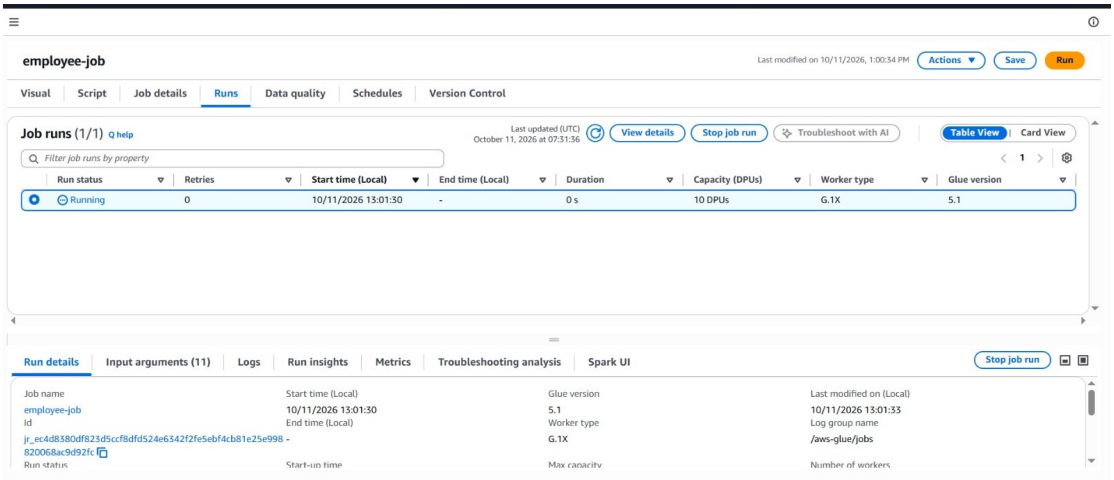
S3 Target Configuration:

S3 target configuration: Parquet format with Snappy compression and the S3 target location

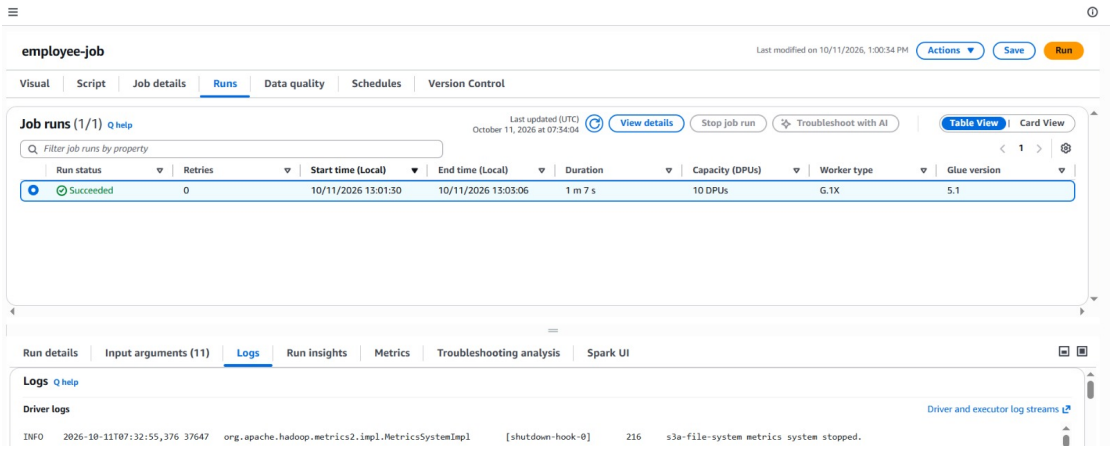


Glue ETL Job Execution:

Job employee-job started: run status Running

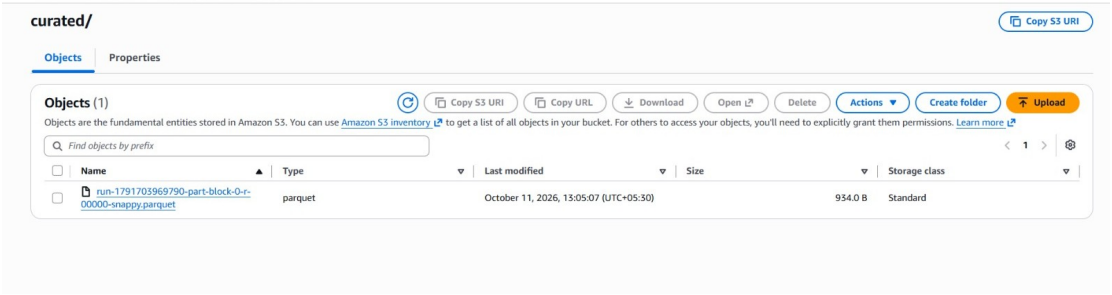


Job employee-job: run status Succeeded



Final S3 Output:

Output in the curated/ folder: a Parquet file generated by the job



Final Explanation : AWS Glue ETL Workflow

1. Amazon S3:

Amazon S3 is the storage layer of this workflow.
We use a bucket with a raw folder and a curated folder.
The employee CSV file is uploaded into the raw folder.
The raw folder keeps the original data as it is.
The curated folder is kept for the cleaned output.

2. Glue Crawler:

A Glue crawler scans the data stored in S3.
Our crawler reads the raw folder using an IAM role.
It identifies the file format and the column names.
It also detects the data type of each column.
These details are written to the Glue Data Catalog.

3. Glue Data Catalog:

The Data Catalog stores details about the data, not the data itself.
It holds the database glue_training_db.
The crawler creates a table inside this database.
Its columns are employee_id, name, department, salary and city.
Glue uses this table as the source of the ETL job.

4. Glue Visual ETL:

Glue Visual ETL is used to build the job by connecting boxes.
The source box reads the table from the Data Catalog.
The next box transforms the data.
The last box writes the result to S3.
The job is saved as employee-job and needs no hand-written code.

5. Transformation:

Drop Fields is the transformation used in this job.
It removes the city column, which is not needed in the output.
The remaining columns move to the next box unchanged.
This gives a smaller and cleaner dataset.
The cleaned data is then sent to the S3 target.

6. S3 Curated Data:

The S3 target box writes the output to the curated folder.
The output format is Parquet with Snappy compression.
When employee-job runs, the status shows Succeeded.
A Parquet file is created in the curated folder.
This file can be used for further analysis and reporting.

WORKFLOW : S3 -> Glue Crawler -> Glue Data Catalog -> Glue Visual ETL -> Transformation -> S3 Curated Data